

Experiment 5

1. Aim

Simulate Page Replacement Algorithms (FIFO, LRU, Optimal) and analyze page faults.

2. Hardware / Software Required

- Windows/Linux/macOS
- Python 3.x
- Python IDLE, VS Code, or any Python-supported IDE

3. Theory

Page replacement is a memory-management technique used in an operating system when a **page fault** occurs and all available frames are occupied. The operating system must select an existing page to remove from memory so that the required page can be loaded.

Page Fault

A **page fault** occurs when a process requests a page that is not currently present in the allocated memory frames.

Page Replacement Algorithms

Algorithm	Description
FIFO (First-In, First-Out)	Replaces the page that has been in memory for the longest time.
LRU (Least Recently Used)	Replaces the page that has not been used for the longest period of time.
Optimal	Replaces the page that will not be used for the longest time in the future. It provides the minimum possible page faults but requires knowledge of future references.

Algorithm:

FIFO

1. Start with empty memory frames.

2. Read each page from the reference string.
3. If the page is already present, no page fault occurs.
4. If the page is absent, generate a page fault.
5. If the frames are full, replace the page that entered first.
6. Repeat until all page references are processed.
7. Count the total page faults.

LRU

1. Start with empty memory frames.
2. Read each page from the reference string.
3. If the page is present, update its recent-use information.
4. If the page is absent, generate a page fault.
5. If the frames are full, replace the least recently used page.
6. Repeat for all page references.
7. Count the total page faults.

Optimal

1. Start with empty memory frames.
2. Read each page from the reference string.
3. If the page is already present, no page fault occurs.
4. If the page is absent, generate a page fault.
5. If frames are full, look at future page references.
6. Replace the page whose next use is farthest in the future, or which will not be used again.
7. Repeat until all pages are processed.
8. Count the total page faults.

4. Procedure

1. Open the Python IDE or terminal and create a new Python program.
2. Define the page reference string and the number of available memory frames.
3. Implement the **FIFO** page replacement algorithm.
4. Implement the **LRU** page replacement algorithm.
5. Implement the **Optimal** page replacement algorithm.
6. Run the program and enter a sequence of page numbers as the **page reference string**.
7. Enter the required number of memory frames.
8. The program checks each requested page and determines whether it is already present in memory.
9. If a page is not present, a **page fault** occurs and the appropriate page replacement algorithm selects a page for replacement.
10. Repeat the process until all page references have been processed.
11. Count and display the total page faults for FIFO, LRU, and Optimal.
12. Compare the page-fault counts to analyze the performance of the three algorithms.

5. Output

```
Welcome X exp_4.py X exp_5.py X
exp_5.py > optimal
1 def fifo(pages, frames):
2     memory = []
3     page_faults = 0
4     pointer = 0
5
6     for page in pages:
7         if page not in memory:
8             page_faults += 1
9
10            if len(memory) < frames:
11                memory.append(page)
12            else:
13                memory[pointer] = page
14                pointer = (pointer + 1) % frames
15
16    return page_faults
17
18 def lru(pages, frames):
19     memory = []
20     page_faults = 0
21
22     for page in pages:
23         if page not in memory:
24             page_faults += 1
25
26            if len(memory) < frames:
27                memory.append(page)
28            else:
29                memory.pop(0)
30                memory.append(page)
31        else:
32            # Move recently used page to the end
33            memory.remove(page)
34            memory.append(page)
35
36    return page_faults
37
```

```
Welcome X exp_4.py X exp_5.py X
exp_5.py > optimal
38 def optimal(pages, frames):
39     memory = []
40     page_faults = 0
41
42     for i in range(len(pages)):
43         page = pages[i]
44
45         if page not in memory:
46             page_faults += 1
47
48             if len(memory) < frames:
49                 memory.append(page)
50             else:
51                 future = pages[i + 1:]
52
53                 indexes = []
54
55                 for p in memory:
56                     if p in future:
57                         indexes.append(future.index(p))
58                     else:
59                         indexes.append(float('inf'))
60
61                 replace_index = indexes.index(max(indexes))
62                 memory[replace_index] = page
63
64     return page_faults
65
66 # Input
67 pages = list(map(int, input("Enter page reference string: ").split()))
68 frames = int(input("Enter number of frames: "))
69 # Calculate page faults
70 fifo_faults = fifo(pages, frames)
71 lru_faults = lru(pages, frames)
72 optimal_faults = optimal(pages, frames)
73 # Display results
74 print("\nPage Replacement Analysis")
75 print("-----")
76 print("FIFO Page Faults      :", fifo_faults)
77 print("LRU Page Faults       :", lru_faults)
78 print("Optimal Page Faults    :", optimal_faults)
```

```
Output  Debug Console  Terminal  Ports

PS D:\OneDrive\Desktop\SOS\codes> python -u "d:\OneDrive\Desktop\SOS\codes\exp_5.py"
Enter page reference string: 7 0 1 2 0 3 0 4 2 3 0 3 2
Enter number of frames: 3

Page Replacement Analysis
-----
FIFO Page Faults      : 10
LRU Page Faults       : 9
Optimal Page Faults   : 7
PS D:\OneDrive\Desktop\SOS\codes> 
```

Conclusion

Thus, the **FIFO, LRU, and Optimal Page Replacement Algorithms** were successfully implemented and analyzed. FIFO replaces the oldest page, LRU replaces the least recently used page, while Optimal replaces the page whose next use is farthest in the future. **Optimal generally produces the minimum number of page faults**, while LRU often performs better than FIFO for practical workloads.